

Universidad Rafael Landívar

Campus San Alberto Hurtado S.J.

Facultad de Ingeniería

Estructura de Datos II

Ing. Manuel Rojas



Universidad
Rafael Landívar
Tradición Jesuita en Guatemala

Manual Técnico:
“Catálogo de una Biblioteca”

Keila Belinda Puac Gómez

1660625

Quetzaltenango, 8 de septiembre de 2026

1. Introducción

Este documento describe la implementación técnica del sistema de gestión de Catálogo de Biblioteca, desarrollado en C# como Proyecto de Laboratorio No. 1 del curso Estructura de Datos II. El sistema administra libros identificados por un código único, gestiona préstamos y devoluciones, y genera reportes ordenados a partir de las estructuras de datos implementadas por el propio estudiante: un Árbol B+, un Min Heap, un Max Heap y una lista enlazada simple.

Ninguna de estas estructuras utiliza colecciones nativas de .NET (List<T>, Dictionary, SortedSet, PriorityQueue, etc.) como mecanismo de almacenamiento u ordenamiento interno. Los únicos usos de tipos nativos son de apoyo auxiliar: arreglos (T[]) administrados manualmente y funciones de lectura de archivos (File.ReadAllLines) y manejo de cadenas.

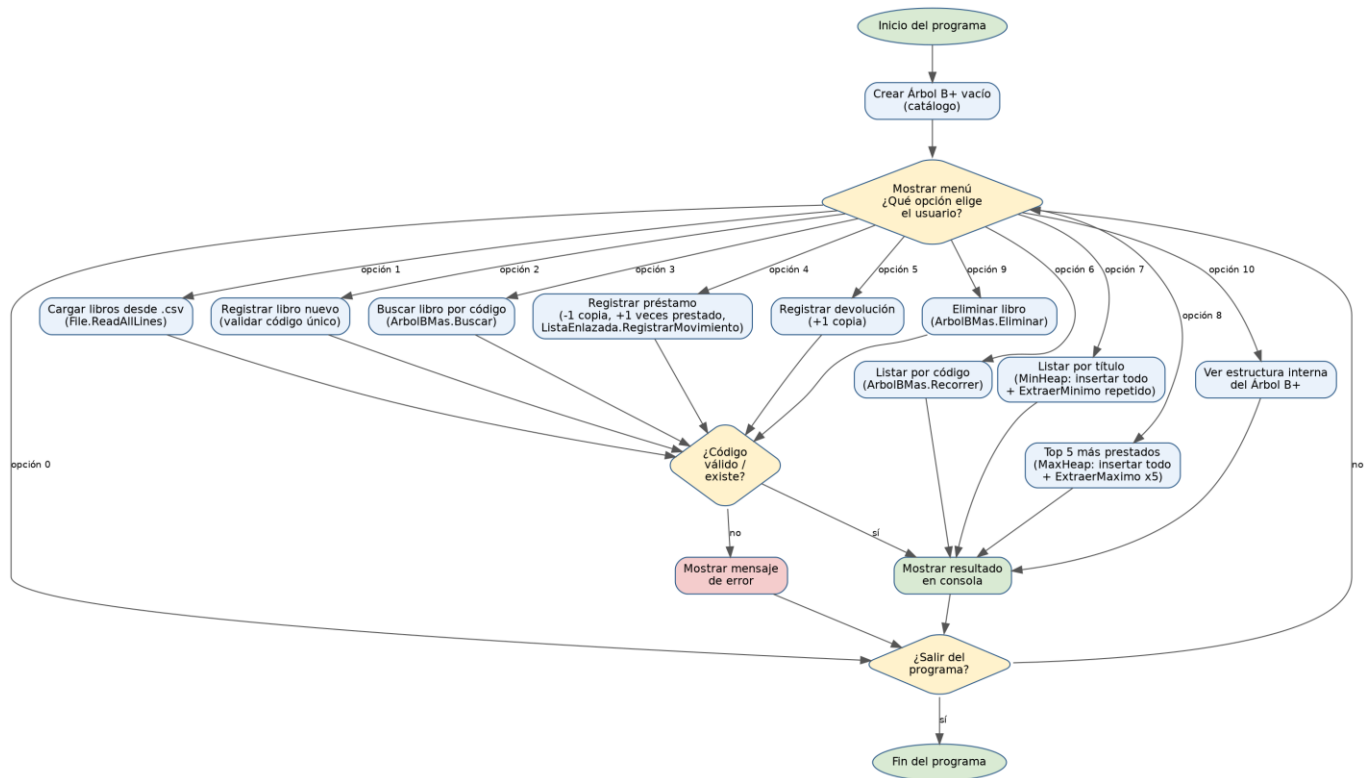
2. Alcance funcional

El programa permite:

- Cargar libros de forma masiva desde un archivo .csv o registrarlos manualmente por menú.
- Buscar un libro específico por su código único.
- Registrar préstamos y devoluciones, actualizando copias disponibles y veces prestado.
- Listar el catálogo completo ordenado por código (recorrido natural del Árbol B+).
- Listar el catálogo completo ordenado alfabéticamente por título (Min Heap).
- Mostrar el Top 5 de libros más prestados (Max Heap).
- Eliminar un libro del catálogo.
- Visualizar la estructura interna del Árbol B+ para fines de estudio y defensa.

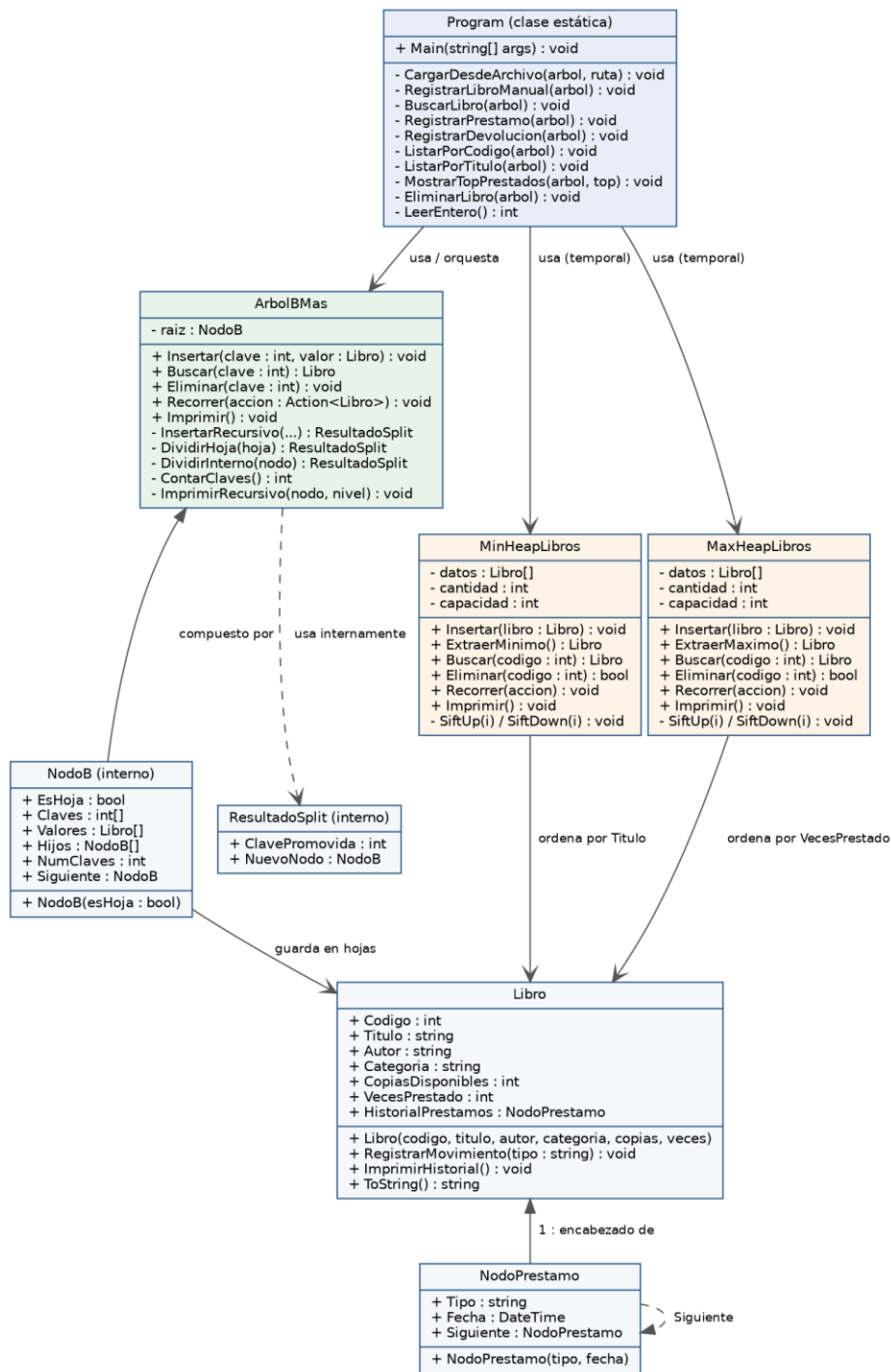
3. Diagrama de flujo general

El siguiente diagrama resume el ciclo de vida del programa: la inicialización del catálogo (Árbol B+ vacío), el bucle del menú principal y el camino que sigue cada una de las opciones hasta mostrar un resultado o un error en consola.



4. Diagrama de clases

El diseño se organiza en dos archivos de código fuente: Estructuras.cs (las tres estructuras de datos obligatorias: ArbolBMas, MinHeapLibros y MaxHeapLibros, además de la clase interna de apoyo NodoB) y Program.cs (las entidades del dominio Libro y NodoPrestamo, y la clase Program con el menú).



5. Justificación de cada estructura de datos

5.1 Árbol B+ — índice principal del catálogo (por Código)

Operación crítica: búsqueda, inserción y recorrido ordenado completo.

Se eligió como la estructura principal porque el enunciado exige que los libros estén indexados por un código único que permita búsquedas rápidas.

A diferencia de un BST o un AVL, un Árbol B+ mantiene TODOS los datos en las hojas, las cuales están enlazadas entre sí; esto permite recorrer el catálogo completo en orden ascendente de código en $O(n)$ sin recursión, ideal para 'generar un listado de todo el catálogo'. Los nodos internos solo contienen claves guía, por lo que el árbol permanece con poca altura (factor de ramificación $ORDEN = 4$), logrando búsquedas e inserciones en $O(\log n)$.

Complejidad: Buscar $O(\log n)$ · Insertar $O(\log n)$ · Eliminar $O(n)$ (ver nota de diseño en 6.3) · Recorrer $O(n)$.

5.2 Min Heap — listado ordenado por Título

Operación crítica: producir el catálogo completo ordenado alfabéticamente.

El catálogo vive indexado por código en el Árbol B+, no por título. Para no reordenar el árbol completo (lo cual violaría su invariante de orden por código), se construye un Min Heap temporal con todos los libros y se aplica Heap Sort: insertar todos $O(n \log n)$ y extraer el mínimo repetidamente $O(n \log n)$.

Se eligió un heap y no un algoritmo de ordenamiento genérico porque el enunciado exige aplicar un Min Heap como estructura obligatoria, y porque un heap es la estructura de datos clásica y más eficiente para esta tarea (mejor que ordenamientos $O(n^2)$ como burbuja o inserción).

Complejidad total de la operación de listado: $O(n \log n)$.

5.3 Max Heap — Top 5 de libros más prestados

Operación crítica: obtener rápidamente los k elementos con mayor 'VecesPrestado'.

Cuando solo se necesitan los k mayores elementos de una colección (y k es pequeño y fijo, como el Top 5), un Max Heap es más eficiente que ordenar todo el catálogo: construir el heap es $O(n)$ y cada extracción del máximo es $O(\log n)$, por lo que obtener el Top 5 cuesta $O(n + 5 \cdot \log n)$ en vez de $O(n \log n)$ de un ordenamiento completo.

Se construye de forma temporal (igual que el Min Heap) a partir de un recorrido del Árbol B+, preservando el árbol como única fuente de verdad del catálogo.

5.4 Lista enlazada simple — historial de préstamos por libro

Operación crítica: registrar movimientos cronológicos y consultarlos.

Cada libro puede acumular un número impredecible de préstamos y devoluciones. Solo se necesita insertar un nuevo movimiento (siempre al inicio, $O(1)$) y recorrer el historial secuencialmente para mostrarlo ($O(n)$).

No se requiere búsqueda por clave, orden distinto al cronológico, ni acceso aleatorio, por lo que un árbol o un heap serían un sobre costo injustificado: la lista enlazada simple es la estructura mínima suficiente para este caso de uso.

6. Decisiones de diseño relevantes

6.1 Orden del Árbol B+ (ORDEN = 4)

Se definió ORDEN = 4 (máximo 4 hijos por nodo interno, máximo 3 claves por nodo). Este valor se eligió por ser pedagógicamente claro (equivalente a un árbol 2-3-4) y suficiente para el volumen de datos típico de un catálogo de biblioteca. Puede ajustarse cambiando una sola constante (NodoB.ORDEN) sin modificar el resto del algoritmo.

6.2 Reserva de espacio 'de más' en los nodos (sobrecarga controlada)

Cada nodo reserva arreglos de tamaño ORDEN (y no ORDEN-1). Esto permite insertar temporalmente una clave adicional en un nodo lleno y decidir DESPUÉS si se divide (split), en vez de tener que verificar el espacio disponible antes de cada inserción. Es una técnica estándar en implementaciones didácticas de árboles B/B+ que simplifica considerablemente el algoritmo de inserción.

6.3 Eliminación por reconstrucción en el Árbol B+

La eliminación 'in-place' de un Árbol B+ (pedir prestado a nodos hermanos o fusionar nodos cuando uno queda con déficit de claves, actualizando separadores en cascada) es considerablemente compleja y propensa a errores sutiles difíciles de depurar. Para este proyecto se optó deliberadamente por una estrategia de eliminación por reconstrucción:

- 1) Se recorre el árbol completo (usando la lista enlazada de hojas) copiando todas las parejas código/libro EXCEPTO la que se va a eliminar.

- 2) Se vacía el árbol (raíz = null).

- 3) Se reinserta cada pareja restante usando el mismo método Insertar ya verificado.

El resultado es exactamente un árbol balanceado y válido — el mismo que produciría el algoritmo in-place — pero con una implementación muchísimo más simple de razonar, probar y defender oralmente. El costo es $O(n \log n)$ en vez de $O(\log n)$ por eliminación, un costo aceptable dado que el catálogo de una biblioteca no se elimina con alta frecuencia ni a gran escala.

6.4 Heaps como estructuras temporales, no como almacenamiento persistente

El Min Heap y el Max Heap NO almacenan el catálogo de forma permanente: se construyen bajo demanda a partir de un recorrido del Árbol B+ (única fuente de verdad) cada vez que el usuario solicita un listado por título o un Top 5, y se descartan al terminar la operación. Esto evita mantener tres copias sincronizadas del catálogo y refleja el uso natural de un heap: una estructura auxiliar para resolver una consulta puntual de orden o de 'los k mejores/peores', no un índice principal.

6.5 Recorrido mediante delegados (Action<Libro>)

El método Recorrer de las tres estructuras recibe un delegado Action<Libro> en lugar de devolver una colección de resultados. Esto permite recorrer e imprimir, filtrar, o insertar en otra estructura sin depender de List<T> ni de ningún otro contenedor nativo de .NET, cumpliendo el requisito de implementación propia.

7. Complejidad algorítmica — resumen

Estructura	Insertar	Buscar	Eliminar	Recorrer / Listar
Árbol B+	$O(\log n)$	$O(\log n)$	$O(n \log n)^*$	$O(n)$
Min Heap	$O(\log n)$	$O(n)^{**}$	$O(n)^{**}$	$O(n \log n)$ para orden completo
Max Heap	$O(\log n)$	$O(n)^{**}$	$O(n)^{**}$	$O(n + k \log n)$ para Top k
Lista enlazada	$O(1)$ (al inicio)	—	—	$O(n)$

8. Estructura de archivos del proyecto

Program.cs -> Libro, NodoPrestamo, clase Program (Main + menú)
Estructuras.cs -> NodoB, ArbolBMas, MinHeapLibros, MaxHeapLibros
<Proyecto>.csproj -> archivo de proyecto de .NET (generado por el IDE)
libros_ejemplo.csv -> datos de ejemplo para la carga dinámica

9. Conclusiones

El sistema cumple con los requisitos técnicos mínimos del proyecto: implementa y aplica un Árbol B+, un Min Heap y un Max Heap desarrollados en su totalidad por el estudiante, cada uno justificado según la operación que resuelve dentro del escenario de Catálogo de biblioteca. Las decisiones de diseño documentadas en la sección 6 priorizan la corrección y la claridad del código sobre la optimalidad absoluta, lo cual es apropiado para el volumen de datos de un catálogo bibliotecario y facilita la comprensión y defensa individual del proyecto.